LIST OF PROGRAMS

Defining an Event-Driven Microservice and Its Boundaries

The following practical programs are prepared from the concepts covered in the uploaded lecture material, including service architecture, message types, messaging patterns, RabbitMQ-based event handling, asynchronous services, process management, service boundaries, and reliable event publication.

S.No.	Program Title	Program Description	Suggested Technology
1	**Develop a Three-Tier Microservice Application**	Implement presentation/API, application/domain, and data layers for a simple Order or Product service.	Spring Boot, REST API, JPA
2	**Develop a Microservice Using Clean Architecture**	Separate domain logic from infrastructure by using ports/interfaces and adapters for API, persistence, and messaging.	Spring Boot, Clean Architecture
3	**Create Command, Event, Document, and Query Message Classes**	Define examples such as CreateOrderCommand, OrderCreatedEvent, OrderDocument, and GetInventoryAvailability.	Java Records / POJOs
4	**Create an OrderCreatedEvent Payload**	Implement an event payload containing event id, correlation id, order id, customer id, amount, and timestamp.	Java, Spring Boot
5	**Publish an Event from Order Service**	After validating and saving an order, publish OrderCreatedEvent to a RabbitMQ exchange using a routing key.	Spring Boot, RabbitMQ
6	**Consume an Event in Inventory Service**	Create an event listener that receives OrderCreatedEvent and performs inventory reservation independently.	Spring Boot, @RabbitListener
7	**Implement a Command Handler for CreateOrderCommand**	Receive a point-to-point command in the owning Order Service and invoke the corresponding domain operation.	Spring Boot, RabbitMQ
8	**Configure RabbitMQ Exchange, Queue, and Binding**	Create a topic exchange, durable queue, and routing-key binding for an event subscriber.	Spring AMQP, RabbitMQ
9	**Implement Send/Receive Messaging Pattern**	Send a command to a single service owner through a queue and process it using competing consumers.	RabbitMQ, Spring Boot
10	**Implement Publish/Subscribe Messaging Pattern**	Publish one domain event and allow Inventory, Notification, Shipping, and Analytics consumers to react independently.	RabbitMQ Topic Exchange
11	**Implement Asynchronous Request/Response Using Correlation ID**	Send a request/command and correlate the later response or completion event without blocking the caller.	Spring Boot, Messaging
12	**Develop a CRUD Service that Publishes Events**	Provide synchronous REST-based CRUD operations and publish an event whenever relevant data changes.	Spring Boot REST, JPA, RabbitMQ
13	**Develop a Fully Asynchronous Microservice**	Receive commands/events from queues or topics and publish resulting events/documents without synchronous service calls.	Spring Boot, RabbitMQ
14	**Implement Retry and Dead-Letter Queue Handling**	Handle consumer failures through retries and route repeatedly failing messages to a dead-letter queue.	RabbitMQ DLQ, Spring AMQP
15	**Implement Idempotent Event Consumer**	Store processed event IDs and prevent duplicate business actions when the same message is delivered more than once.	Spring Boot, Database
16	**Implement a Process Manager / Orchestrator**	Listen for PaymentApprovedEvent and issue ShipOrderCommand to coordinate a multi-step order workflow.	Spring Boot, RabbitMQ
17	**Implement Transactional Outbox Pattern**	Save the business transaction and outbox event in the same database transaction and publish the event asynchronously.	Spring Boot, JPA, RabbitMQ
18	**Design an E-Commerce System Using Bounded Contexts**	Create separate Catalog, Inventory, Order, Payment, Shipping, Notification, and Analytics service boundaries.	DDD, Microservices
19	**Implement Aggregate-Based Order Service**	Model an Order aggregate and enforce rules that must remain consistent inside one transaction before publishing events.	DDD, Spring Boot
20	**Build a GDPR-Aware Event Publishing Example**	Minimize sensitive event data and demonstrate encryption or key-based protection for personal information in durable events.	Spring Boot, Security

Recommended sequence: Programs 1–8 can be used as basic exercises; Programs 9–16 as intermediate exercises; Programs 17–20 as advanced microservices exercises.